

ASSIGNMENT 9.1

Name: Kasuri Rohith

Batch: 23

Hall no: 2303a510i2

Problem 1:

Consider the following Python function:

```
def find_max(numbers):  
    return max(numbers)
```

Task:

- Write documentation for the function in all three formats:

(a) Docstring

(b) Inline comments

(c) Google-style documentation

- Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
- Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

Code:

```

demo1.py > ...
1  #(a) Docstring
2  def find_max(numbers):
3      """
4      Return the largest number from a list of numbers.
5      """
6      return max(numbers)
7
8  #(b) Inline Comments
9  def find_max(numbers):
10     # Use the built-in max function to find the highest value in the sequence
11     return max(numbers) # Returns the maximum value found
12
13  #(c) Google-Style Documentation
14  def find_max(numbers):
15     """
16     Return the largest number from a list of numbers.
17
18     Args:
19         numbers (list): A list of numerical values.
20     Returns:
21         The largest number in the list.
22     """
23     return max(numbers)
24
25  numbers = [3, 1, 4, 1, 5, 9]
26  max_value = find_max(numbers)
27  print(max_value) # Output: 9

```

Output:

```

PS C:\Users\rohit\OneDrive\Desktop\lat.py> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/lat.py/demo1.py
9
PS C:\Users\rohit\OneDrive\Desktop\lat.py>

```

Problem 2: Consider the following Python function:

```

def login(user, password, credentials):
    return credentials.get(user) == password

```

Task:

1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

code:

```

31 #docstring
32 def login(user, password, credentials):
33     """
34     Verify if the provided password matches the stored credential for a user.
35     """
36     return credentials.get(user) == password
37
38 #inline
39 def login(user, password, credentials):
40     # This won't show up in pydoc retrieval!
41     return credentials.get(user) == password
42
43 #google style
44 def login(user, password, credentials):
45     """
46     Checks user credentials against a dictionary of authorized users.
47
48     Args:
49         user (str): The username attempting to log in.
50         password (str): The plaintext password provided by the user.
51         credentials (dict): A dictionary mapping usernames (str) to passwords (str).
52
53     Returns:
54         bool: True if the password matches the stored value, False otherwise.
55     """
56     return credentials.get(user) == password

```

Output:

```

PS C:\Users\rohit\OneDrive\Desktop\lat.py> python -m pydoc demo1
Help on module demo1:

NAME
    demo1

DESCRIPTION
    # #(a) Docstring
    # def find_max(numbers):
    #     """

NAME
    demo1

DESCRIPTION
    # #(a) Docstring
    # def find_max(numbers):
    #     """

DESCRIPTION
    # #(a) Docstring
    # def find_max(numbers):
    #     """
    #     Return the largest number from a list of numbers.
    #     """
    #     return max(numbers)

FUNCTIONS
    login(user, password, credentials)
        Checks user credentials against a dictionary of authorized users.

        Args:
            user (str): The username attempting to log in.
            password (str): The plaintext password provided by the user.
            credentials (dict): A dictionary mapping usernames (str) to passwords (str).

        Returns:
            bool: True if the password matches the stored value, False otherwise.

FILE
    c:\users\rohit\onedrive\desktop\lat.py\demo1.py

```

Problem 3: Calculator (Automatic Documentation Generation)

Task: Design a Python module named `calculator.py` and demonstrate automatic documentation generation.

Instructions:

1. Create a Python module `calculator.py` that includes the following functions, each written with appropriate docstrings:
 - o `add(a, b)` – returns the sum of two numbers
 - o `subtract(a, b)` – returns the difference of two numbers
 - o `multiply(a, b)` – returns the product of two numbers
 - o `divide(a, b)` – returns the quotient of two numbers
2. Display the module documentation in the terminal using Python's documentation tools.
3. Generate and export the module documentation in HTML format using the `pydoc` utility, and open the generated HTML file in a web browser to verify the output.

Code:

```
1  """
2  Calculator Module
3  This module provides basic arithmetic operations:
4  addition, subtraction, multiplication, and division.
5  """
6
7  def add(a, b):
8      """
9      Return the sum of two numbers.
10     Args:
11         a (int or float): First number
12         b (int or float): Second number
13
14     Returns:
15         int or float: Sum of a and b
16     """
17     return a + b
18
19  def subtract(a, b):
20      """
21      Return the difference of two numbers.
22     Args:
23         a (int or float): First number
24         b (int or float): Second number
25
26     Returns:
27         int or float: Difference of a and b
28     """
29     return a - b
30
31  def multiply(a, b):
32      """
33      Return the product of two numbers.
34     Args:
35         a (int or float): First number
36         b (int or float): Second number
37     Returns:
38         int or float: Product of a and b
39     """
40     return a * b
41
42  def divide(a, b):
43      """
44      Return the division of two numbers.
45     Args:
46         a (int or float): First number
47         b (int or float): Second number
48     Returns:
49         int or float: Result of a divided by b
50     """
51     return a / b
```

[index](#)

lab_9_1 [c:\users\2303a\onedrive\desktop\ai\lab_9_1\pr](#)

Calculator Module

Provides basic arithmetic operations such as addition, subtraction, multiplication, and division.

Functions

add(a, b)
Return the sum of two numbers.

Args:
a (int/float): First number
b (int/float): Second number

Returns:
int/float: Sum of a and b

divide(a, b)
Return the quotient of two numbers.

Args:
a (int/float): Numerator
b (int/float): Denominator

Returns:
float: Result of division

Raises:
ValueError: If b is zero

multiply(a, b)
Return the product of two numbers.

Args:
a (int/float): First number
b (int/float): Second number

Problem 4: Conversion Utilities Module

Task:

1. Write a module named `conversion.py` with functions:

```
o decimal_to_binary(n)
```

o `binary_to_decimal(b)`

o `decimal_to_hexadecimal(n)`

2. Use Copilot for auto-generating docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a browser.

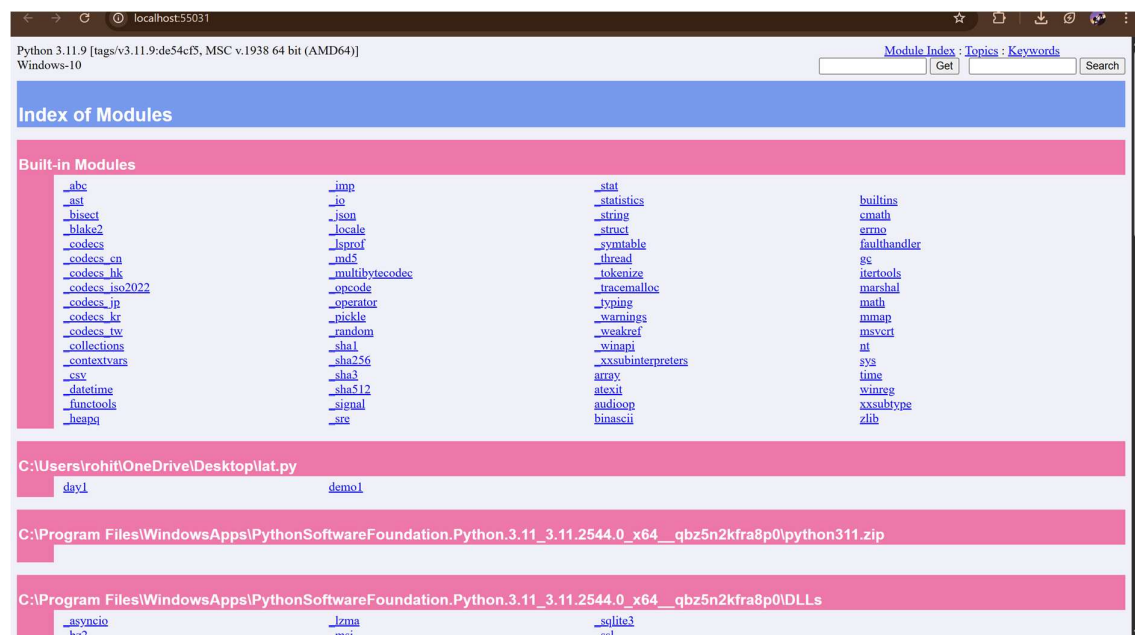
Code:

```
68 """
69 Conversion Utilities Module
70 This module provides functions to convert numbers
71 between decimal, binary, and hexadecimal formats.
72 """
73
74 def decimal_to_binary(n):
75     """
76     Convert a decimal integer to its binary representation.
77
78     Args:
79         n (int): Decimal number
80
81     Returns:
82         str: Binary representation of the number
83     """
84     return bin(n)[2:]
85
86
87 def binary_to_decimal(b):
88     """
89     Convert a binary string to its decimal equivalent.
90
91     Args:
92         b (str): Binary number as a string
93
94     Returns:
95         int: Decimal representation of the binary number
96     """
97     return int(b, 2)
98
99
100 def decimal_to_hexadecimal(n):
101     """
102     Convert a decimal integer to its hexadecimal representation.
103
104     Args:
105         n (int): Decimal number
106
107     Returns:
108         str: Hexadecimal representation of the number
109     """
110     return hex(n)[2:]
```

```

PS C:\Users\2303a\OneDrive\Desktop\AI> python -m pydoc -w conversion
wrote conversion.html
PS C:\Users\2303a\OneDrive\Desktop\AI> python -m pydoc -p 10031
Server ready at http://localhost:10031/
wrote conversion.html
PS C:\Users\2303a\OneDrive\Desktop\AI> python -m pydoc -p 10031
Server ready at http://localhost:10031/
Server ready at http://localhost:10031/
Server commands: [b]rowser, [q]uit
server> 

```



Problem 5 – Course Management Module

Task:

1. Create a module course.py with functions:

o add_course(course_id, name, credits)

o remove_course(course_id)

o get_course(course_id)

2. Add docstrings with Copilot.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a browser.

Code:

```
demo1.py > get_course
1  """
2  Course Management Module
3  This module provides functionality to manage courses,
4  including adding, removing, and retrieving course details.
5  """
6
7  courses = {}
8
9  def add_course(course_id, name, credits):
10     """
11     Add a new course to the course dictionary.
12
13     Args:
14         course_id (str): Unique identifier of the course.
15         name (str): Name of the course.
16         credits (int): Number of credits assigned to the course.
17
18     Returns:
19         None
20     """
21     courses[course_id] = {
22         "name": name,
23         "credits": credits
24     }
25
26
27  def remove_course(course_id):
28     """
29     Remove a course from the system.
30
31     Args:
32         course_id (str): Unique identifier of the course.
33
34     Returns:
35         dict or None: Removed course details if course exists,
36         otherwise None.
37     """
38     return courses.pop(course_id, None)
39
40
41  def get_course(course_id):
42     """
43     Retrieve course details by course ID.
44
45     Args:
46         course_id (str): Unique identifier of the course.
47
48     Returns:
49         dict or None: Course details if found, otherwise None.
50     """
51     return courses.get(course_id)
```

```
PS C:\Users\rohit\OneDrive\Desktop\lat.py> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/lat.py/demo1.py
PS C:\Users\rohit\OneDrive\Desktop\lat.py> python -m pydoc -p 10031
Server ready at http://localhost:10031/
Server commands: [b]rowser, [q]uit
server>
```

Python 3.13.5 [main, MSC v.1929 64 bit (AMD64)]
Windows-11

[Module Index](#) : [Topics](#) : [Keywords](#)

Get

Search

Index of Modules

Built-in Modules

_abc	_interechannels	_stat	
_ast	_interneques	_statistics	
_bisect	_intereaters	_strins	
_blake2	_io	_strins	cmath
_codecs	_json	_suggestions	errno
_codecs_cn	_locale	_symtable	faulthandler
_codecs_hk	_lsprof	_sysconfig	gc
_codecs_iso2022	_md5	_thread	itertools
_codecs_jp	_multibytecodes	_tokenize	marshal
_codecs_kr	_opcode	_tracemalloc	math
_codecs_tw	_operator	_typing	mmap
_collections	_pickle	_warnings	mover
_contextvars	_random	_weakref	nt
_csv	_sha1	_winapi	os
_datetime	_sha2	_array	time
_functools	_sha3	_atexit	winreg
_heapq	_signal	_binascii	xxsubtype
_imp	_sre	_builtins	zlib

C:\Users\2303a\OneDrive\Desktop\AI

18-2-26	LAB_TEST_1	conversion	lab_9_1
2_Question	Lab_8_13	course	math_utils
Docexample	asa	doctest	test_18-2-26
Docstring	calculator	documentationexample	

C:\Users\2303a\miniconda3\python313.zip